

GitHub 協作操作指南

兩位同學共同開發 calorie-vue 專案

推薦工作方式

加入 Collaborator + 每人使用自己的功能分支 + Pull Request

審查後合併。這樣可以同時開發，又能降低互相覆蓋程式碼的風險。

專案網址	https://github.com/lunachiu2026/calorie-vue
主要分支	main
使用技術	Vue 3 + Vite + GitHub Pages

快速流程

邀請同學 -> Clone 專案 -> 建立分支 -> 修改與測試 -> Commit -> Push -> Pull Request -> Merge

一、專案擁有者：邀請同學

- 1 打開 GitHub 專案
前往 [calorie-vue Repository](#)，確認自己已登入專案擁有者帳號。
- 2 進入權限設定
點選 **Settings**，在左側 **Access** 區域選擇 **Collaborators**。
- 3 新增協作者
點選 **Add people**，輸入同學的 GitHub 使用者名稱或 Email。
- 4 接受邀請
同學必須從 GitHub 通知或 Email 接受邀請，接受後才可以 Push 程式碼。

權限提醒

Collaborator 可以修改程式碼及建立 Pull Request。不要分享自己的 GitHub 密碼或存取權杖；每個人都使用自己的帳號。

二、同學第一次下載專案

同學在自己的電腦開啟 Terminal、PowerShell 或 VS Code 終端機，依序執行：

第一次 Clone

```
git clone https://github.com/lunachiu2026/calorie-vue.git
cd calorie-vue
npm install
npm run dev
```

正常結果

執行 `npm run dev` 後，終端機會顯示本機網址。用瀏覽器開啟該網址即可確認專案是否正常。

三、每個人建立自己的功能分支

不要兩個人都直接在 main 上開發。每個功能各自建立分支，完成後再合併。

同學負責會員頁面

```
git switch main
git pull origin main
git switch -c feature/member-page
```

你負責首頁

```
git switch main
git pull origin main
git switch -c feature/homepage
```

分支命名建議

使用 feature/功能名稱，例如 feature/login、feature/booking、feature/mobile-menu。名稱使用英文、小寫及連字號，較容易辨識。

四、修改完成後提交與上傳

檢查、提交與 Push

```
git status
git add .
git commit -m "完成會員頁面"
git push -u origin feature/member-page
```

第一次 Push 使用 -u 建立遠端追蹤。之後同一分支再次上傳，只需要：

同一分支後續上傳

```
git add .
git commit -m "調整會員頁面樣式"
git push
```

五、建立 Pull Request 並合併

1

建立 Pull Request

Push 後打開 GitHub，點選 Compare & pull request。

2

確認合併方向

base 選 main，compare 選同學的功能分支，例如 feature/member-page。

3

寫清楚修改內容

標題說明完成的功能，內容列出修改檔案、測試方式及仍需注意的事項。

4

檢查程式

另一位同學檢查 Files changed，確認沒有刪除不相關程式，也確認 GitHub Actions 通過。

5

合併

確認沒問題後按 Merge pull request。main 更新後，GitHub Pages

會依工作流程重新部署。

Pull Request 說明範例

可以直接參考

Title: 完成會員登入頁面

Changes:

- 更新登入按鈕樣式
- 新增登入錯誤提示
- 完成手機版排版

Test:

- npm run build
- 瀏覽器實際登入測試

合併前不要急

Pull Request 是你們互相檢查程式的地方。確認畫面、功能及 npm run build 都正常後再 Merge。

六、每天開始工作前先同步

main 可能已經包含另一位同學的新功能，因此每天開始前先更新：

```
更新 main  
git switch main  
git pull origin main
```

如果要建立新的功能分支，必須從最新的 main 建立：

```
建立下一個功能分支  
git switch -c feature/new-feature
```

七、遇到衝突怎麼辦

如果兩個人修改同一個檔案的同一段程式，Git 可能顯示 CONFLICT。請先停止 Push，不要刪除整個檔案，也不要使用 `git reset --hard`。

```
先把 main 合併進自己的分支  
git switch feature/member-page  
git merge main
```

VS Code 會標示 Current Change 與 Incoming Change。和同學確認要保留哪些內容，修改完成後再提交：

```
解決衝突後  
git add .  
git commit -m "解決合併衝突"  
git push
```

避免衝突的方法

先分配工作範圍；避免同時修改同一個 Vue 檔案；開始前先 `git pull`；小量多次 Commit；使用 Pull Request 檢查。

八、常用指令速查表

目的	指令
查看目前狀態	git status
查看目前分支	git branch
切換 main	git switch main
取得最新版本	git pull origin main
建立功能分支	git switch -c feature/name
加入修改	git add .
建立提交	git commit -m "修改說明"
上傳分支	git push -u origin feature/name

合作檢查清單

- ☐ 同學已接受 Collaborator 邀請
- ☐ 每個人使用自己的分支
- ☐ 開始前已更新 main
- ☐ Commit 訊息清楚描述修改內容
- ☐ npm run build 測試成功
- ☐ Pull Request 已由另一位同學檢查
- ☐ 合併後 GitHub Actions 部署成功

記住這句話

先 Pull、分支開發、小量 Commit、Pull Request 檢查、確認後 Merge。

官方參考資料

GitHub Docs - Inviting collaborators to a personal repository